

Online Reading Tracker

Project summary for a Django + Django REST Framework reading application.

Online Reading Tracker is a full-stack Django application for browsing books, reading them in the browser, tracking personal progress, and writing reviews. The project includes a template-based web interface, a DRF API, token authentication, role-based catalog permissions, and a Postman collection with saved example responses.

Project Overview

User Features

- Register, log in, log out, edit profile, and manage settings.
- Browse the catalog, search by title, and filter by genre.
- Open a book detail page and read a book in the browser.
- Save current page, reading status, and personal library items.
- Create public reviews with 1-5 star ratings.

Admin/Staff Features

- Create, update, and delete catalog books and genres.
- Manage users, books, progress, and reviews in Django Admin.
- Protect catalog-changing actions from regular users.
- Use uploaded covers and avatars during local development.
- Keep public read-only endpoints available without login.

Team Contributions

Team member	Main responsibility	Completed work visible in commits/tasks
Saken Ayan	Frontend, templates, CSS, navigation	Implemented and improved the base layout, auth/profile pages, book list page, book detail page, navigation, catalog styling, responsive cards, and user-facing interface polish.
Askarova Akbota	Models, forms, Django views, reader/tracker flow	Added library and tracker application files, book/progress/review models and forms, catalog and tracker views, user forms/views updates, and the book reader feature.
Bisenev Ualikhan	DRF API, serializers, token auth, Postman, docs, integration	Initialized and configured the Django project, added DRF API endpoints, serializers, token authentication, protected permissions, Postman collection/environment, README/API docs, ERD updates, reader progress resume, static file cleanup, and final integration fixes.

Git history also includes merge commits by [Gore_Pisatel](#) for integrating feature branches and pull requests.

Tech Stack

Layer	Technology	Purpose
-------	------------	---------

Backend	Python, Django	Models, views, forms, routing, admin panel, and authentication.
API	Django REST Framework, DRF Token Auth	JSON endpoints, serializers, protected requests, and Postman testing.
Frontend	Django Templates, HTML, CSS, JavaScript	Catalog pages, reader page, profile/settings screens, and theme behavior.
Storage	SQLite, Media files	Local database plus uploaded covers and avatars in development.

Project Structure

```

online_reading_tracker/
├── apps/
│   ├── library/      models, forms, catalog views, book/genre API
│   ├── tracker/      reading progress, reviews, tracker API
│   └── users/         custom user, settings, auth/profile API
├── settings/         base/dev/prod configuration and root urls
├── templates/        base layout, library, tracker, and users pages
├── static/           CSS, JavaScript, logos, and default images
├── media/            uploaded covers and avatars for local development
├── docs/             ERD image, supporting docs, and this PDF summary
├── postman/          collection and local environment for API testing
├── manage.py
├── requirements.txt
└── README.md

```

Data Model

Model	Role in the system
CustomUser , UserSettings	Authentication, profile data, role checks, privacy/theme/notification settings.
Genre , Book	Book catalog, authors, descriptions, reader text, covers, total page calculation.
ReadingProgress , Review	User-specific reading status/current page and public book ratings/reviews.

API Implementation

The API is implemented with Django REST Framework. Protected endpoints use DRF token authentication with the header `Authorization: Token <token>`. User-owned objects are created from `request.user`, so clients cannot assign another user manually.

Area	Endpoints
Users	POST /api/users/register/ , POST /api/users/login/ , POST /api/users/logout/ , GET /api/users/me/ , PATCH /api/users/me/update/
Books and genres	GET /api/books/ , GET /api/books/<id>/ , POST /api/books/create/ , PATCH /api/books/<id>/update/ , DELETE /api/books/<id>/delete/ , GET /api/books/genres/ , POST /api/books/genres/create/

Tracker	<code>GET /api/tracker/progress/</code> , <code>POST /api/tracker/progress/create/</code> , <code>PATCH /api/tracker/progress/<id>/update/</code> , <code>GET /api/tracker/reviews/</code> , <code>POST /api/tracker/reviews/create/</code>
---------	---

Permissions and Security

- Public users can read book lists, book details, genre lists, and review lists.
- Authenticated users can view/update their profile, create progress, update only their own progress, and create reviews.
- Book and genre create/update/delete endpoints are restricted to staff, admin, or moderator users.
- Progress and review creation uses `request.user` , so the client cannot assign another owner manually.
- When a user tries to update another user's progress, the API searches by both `pk` and `user=request.user` .

Postman Collection

The repository includes a Postman collection and local environment in `postman/` . The collection covers user authentication, public book/review requests, protected profile and tracker requests, and staff-only book/genre management requests. The login request saves the returned token into the environment variable `token` .

- Collection: `postman/Online Reading Tracker API.postman_collection.json`
- Environment: `postman/online-reading-tracker.postman_environment.json`
- Base URL: `http://127.0.0.1:8000`

Recommended Demo Flow

1. Start the server and open the book catalog in the browser.
2. Show search, genre filtering, a book detail page, and the browser reader.
3. Log in as a normal user and show My Library, progress update, and review creation.
4. In Postman, run `Login` and show that `token` is saved automatically.
5. Run protected requests: current user, progress list/create/update, and create review.
6. Show that catalog management endpoints require a staff/admin/moderator token.

How To Run

- Install dependencies from `requirements.txt` .
- Run migrations with `python manage.py migrate` .
- Start the server with `python manage.py runserver` .
- Import the Postman collection and environment, run login, then test protected endpoints.